

Лабораторная работа №5

Исследование задачи обедающих философов средствами сетей Петри

true

today

Информация

Докладчик

- Авдадаев Джамал Геланиевич
- Российский университет дружбы народов им. П. Лумумбы
- 1032230169@rudn.ru

Вводная часть

Цель работы

- Реализовать сети Петри для задачи обедающих философов: классическую и с арбитром
- Научиться обнаруживать тупики (deadlock) в модели
- Визуализировать динамику маркировки и провести параметрическое исследование

Объект и предмет исследования

- **Объект:** сеть Петри для задачи обедающих философов
- **Предмет:** условия возникновения тупика и динамика маркировки в классической сети и сети с арбитром

Математическая постановка

- Сеть Петри: позиции Think_i , Hungry_i , Eat_i , Fork_i
- Матрица инцидентности задаёт переходы между состояниями философов и вилок

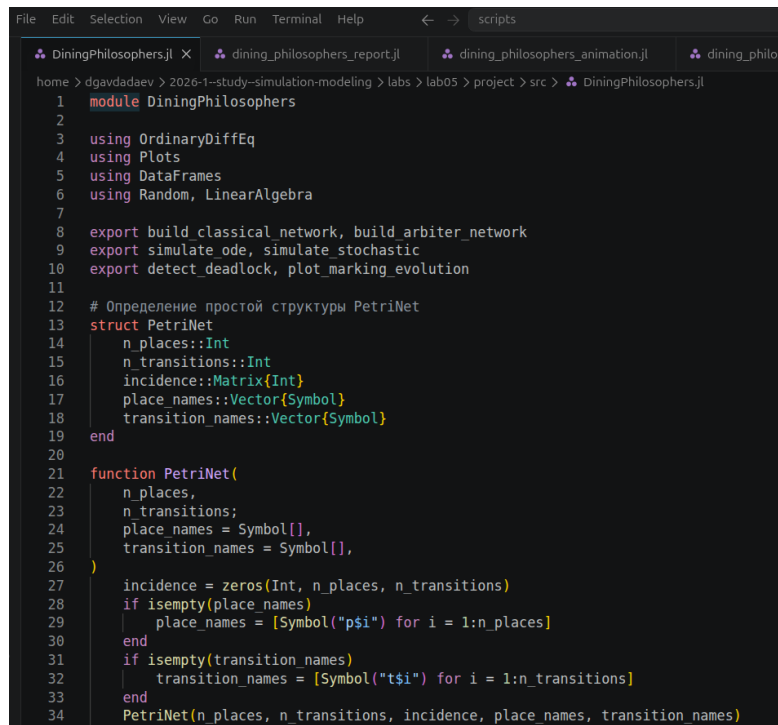
- Классическая сеть допускает одновременный захват соседних вилок → возможен deadlock
- Сеть с арбитром ограничивает число одновременно «голодных» философов

Используемые инструменты

- Julia 1.12.6, DrWatson v2.19.1
- OrdinaryDiffEq, Plots v1.41.6, DataFrames v1.8.2, CSV v0.10.16
- Literate.jl для литературного программирования
- Quarto / Jupyter Notebook

Реализация модели

Модуль DiningPhilosophers



```

File Edit Selection View Go Run Terminal Help ← → scripts
DiningPhilosophers.jl x dining_philosophers_report.jl dining_philosophers_animation.jl dining_philosophers.jl
home > dgavdadaev > 2026-1--study-simulation-modeling > labs > lab05 > project > src > DiningPhilosophers.jl
1 module DiningPhilosophers
2
3 using OrdinaryDiffEq
4 using Plots
5 using DataFrames
6 using Random, LinearAlgebra
7
8 export build_classical_network, build_arbiter_network
9 export simulate_ode, simulate_stochastic
10 export detect_deadlock, plot_marking_evolution
11
12 # Определение простой структуры PetriNet
13 struct PetriNet
14     n_places::Int
15     n_transitions::Int
16     incidence::Matrix{Int}
17     place_names::Vector{Symbol}
18     transition_names::Vector{Symbol}
19 end
20
21 function PetriNet(
22     n_places,
23     n_transitions;
24     place_names = Symbol[],
25     transition_names = Symbol[],
26 )
27     incidence = zeros{Int, n_places, n_transitions}
28     if isempty(place_names)
29         place_names = [Symbol("p$i") for i = 1:n_places]
30     end
31     if isempty(transition_names)
32         transition_names = [Symbol("t$i") for i = 1:n_transitions]
33     end
34     PetriNet(n_places, n_transitions, incidence, place_names, transition_names)

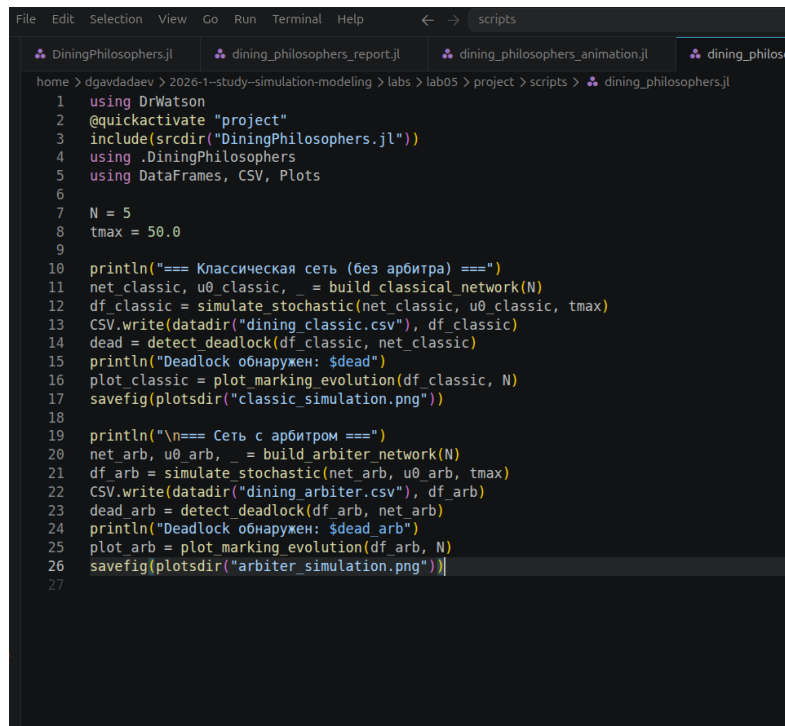
```

Figure 1: Структура PetriNet и экспортируемые функции модуля

- PetriNet: places, transitions, матрица инцидентности

- Функции: `build_classical_network`, `build_arbiter_network`, `simulate_stochastic`, `detect_deadlock`, `plot_marking_evolution`

Базовый скрипт: $N = 5$



```

1 using DrWatson
2 @quickactivate "project"
3 include(sourcedir("DiningPhilosophers.jl"))
4 using .DiningPhilosophers
5 using DataFrames, CSV, Plots
6
7 N = 5
8 tmax = 50.0
9
10 println("=== Классическая сеть (без арбитра) ===")
11 net_classic, u0_classic, _ = build_classical_network(N)
12 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
13 CSV.write(datadir("dining_classic.csv"), df_classic)
14 dead = detect_deadlock(df_classic, net_classic)
15 println("Deadlock обнаружен: $dead")
16 plot_classic = plot_marking_evolution(df_classic, N)
17 savefig(plotsdir("classic_simulation.png"))
18
19 println("\n=== Сеть с арбитром ===")
20 net_arb, u0_arb, _ = build_arbiter_network(N)
21 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
22 CSV.write(datadir("dining_arbiter.csv"), df_arb)
23 dead_arb = detect_deadlock(df_arb, net_arb)
24 println("Deadlock обнаружен: $dead_arb")
25 plot_arb = plot_marking_evolution(df_arb, N)
26 savefig(plotsdir("arbiter_simulation.png"))
27

```

Figure 2: `dining_philosophers.jl`: базовый эксперимент

- Классическая сеть и сеть с арбитром, $N = 5$, $t_{\max} = 50.0$
- Результаты → `dining_classic.csv`, `dining_arbiter.csv`
- Проверка `detect_deadlock` для обоих вариантов

Базовый эксперимент ($N = 5$)

Результат: deadlock в классической сети

- Классическая сеть: Deadlock обнаружен: `true`
- Сеть с арбитром: Deadlock обнаружен: `false`

Классическая сеть: динамика маркировки

- Моделирование останавливается на $t \approx 2.6$

```

julia> include("scripts/dining_philosophers.jl")
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/dgavdadaev/2026-1--study--simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"

```

Figure 3: Deadlock обнаружен: true (классика) / false (арбитр)

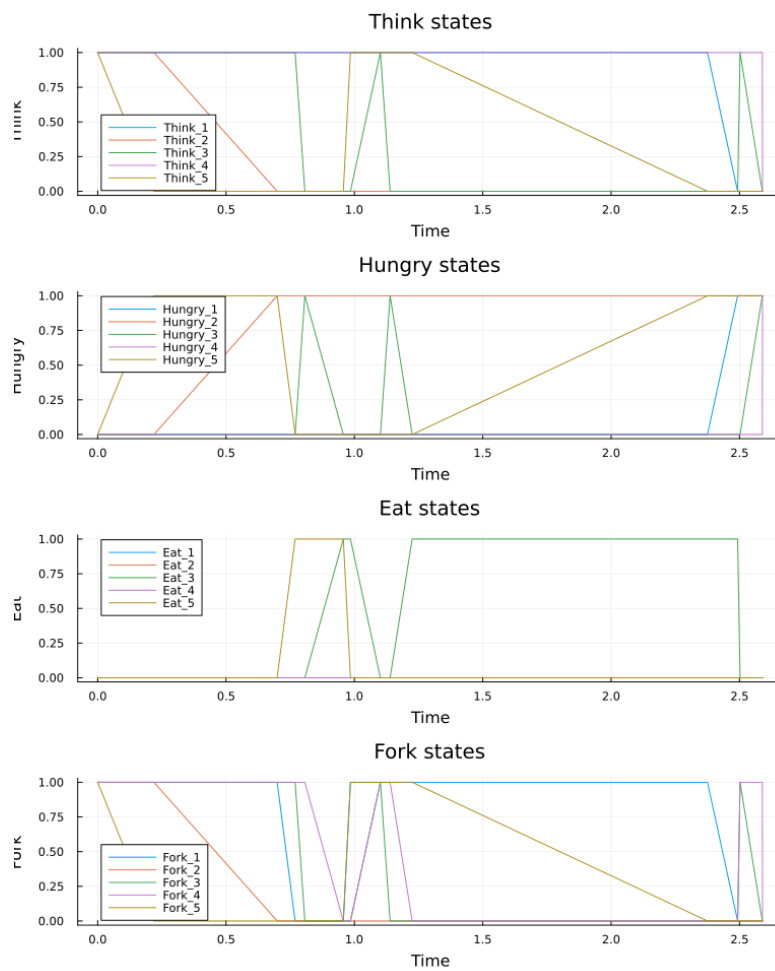


Figure 4: classic_simulation.png

- Поесть успевает только философ 3 (два коротких импульса)
- К $t \approx 1.0$ четыре вилки (Fork_1-Fork_4) зафиксированы на 0

Сеть с арбитром: динамика маркировки

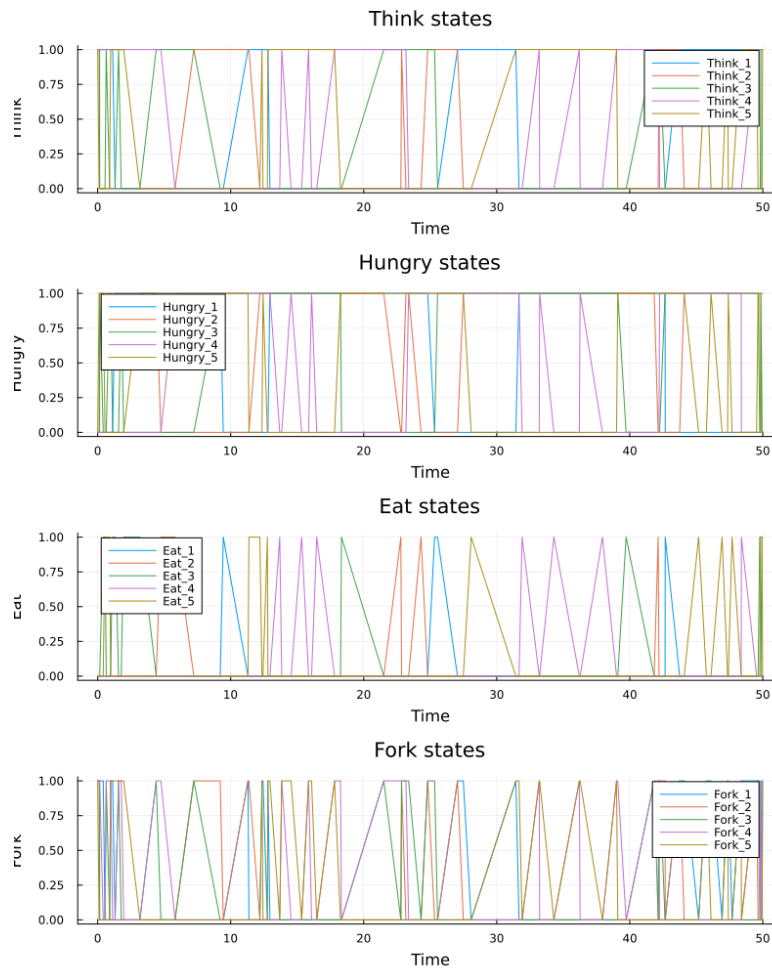


Figure 5: arbiter_simulation.png

- Моделирование идёт весь интервал $t \in [0, 50]$
- Все 5 философов регулярно переходят в состояние «ест»
- Вилки активно переключаются — нет захваченных навсегда ресурсов

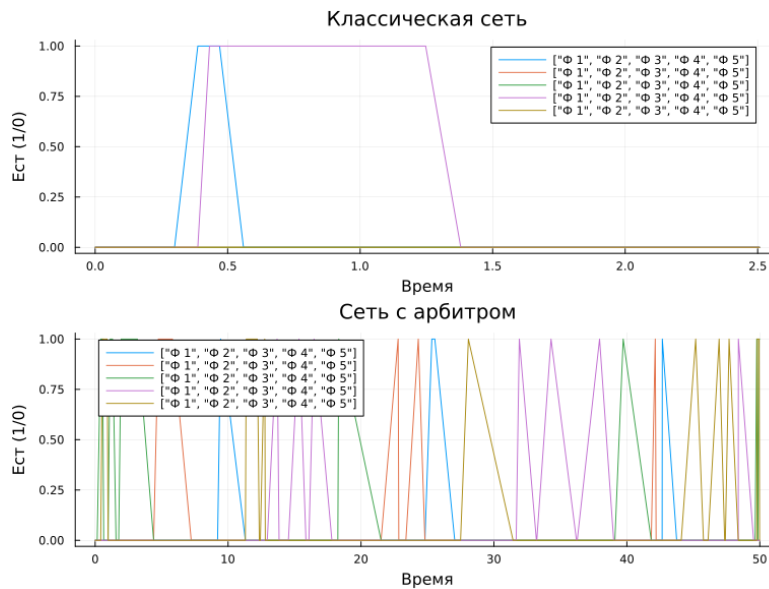


Figure 6: final_report.png: состояния «ест» для классики и арбитра

Итоговый сравнительный график

- Классика ($t \in [0, 2.5]$): только 2 импульса еды, затем тупик
- Арбитр ($t \in [0, 50]$): регулярные импульсы еды у всех 5 философов

Анимация и литературное программирование

Анимация маркировки ($N = 3$)

- $N = 3$, $t_{\max} = 30.0$, `Random.seed!(123)`
- Начальное состояние: все философы Think, все вилки свободны (`Fork_1-3 = 1`)
- Сохранено в `philosophers_simulation.gif`, `fps = 5`

Литературные версии скриптов

- Преобразованы: базовый скрипт, анимация, параметрическое исследование, отчёт
- Сгенерированы Jupyter Notebook (.ipynb) и Quarto-документы (.qmd / .html)

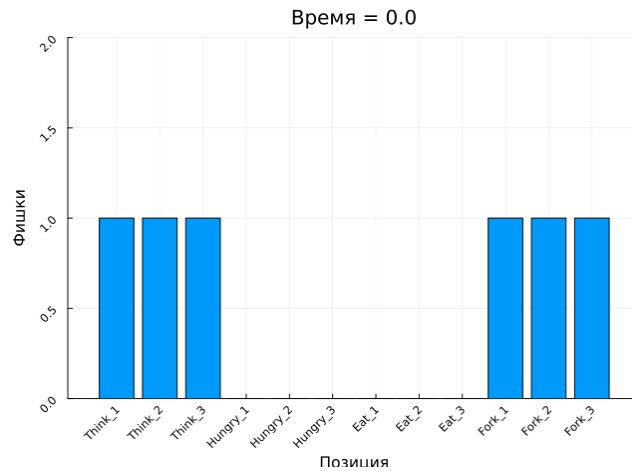


Figure 7: philosophers_simulation.gif: начальный кадр, Время = 0.0

```

diningPhilosophers.jl X dining_philosophers_report.jl dining_philosophers_iterate.jl dining_philosophers_animation_iterate.jl dining_philosophers_report.jl
> dgasdaev ~/2026-1-study-simulation-modeling/labs/lab05/project/src/diningPhilosophers.jl dining_philosophers_iterate.jl
# Исследование задачи обедающих философов средствами сетей Петри
#
# В работе рассматриваются две сети Петри для задачи
# обедающих философов.
# ## импорт библиотек и пакетов
using DrWatson
@quickactivate "project"
include(scrdir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
## Параметры моделирования
N = 5
tmax = 50.0

### Рассматривается классическая сеть (без арбитра)
println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))

### И вторая сеть: сеть с арбитром
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = plot_marking_evolution(df_arb, N)
## сохранение
savefig(plotsdir("arbiter_simulation.png"))

```

Figure 8: dining_philosophers_iterate.jl

Сгенерированные Notebook и Quarto-документы

```
julia> Literate.markdown("scripts/dining_philosophers_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from ~/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_iterate.jl
[ Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_iterate.qmd
~/home/dgavdadev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_iterate.qmd

julia> Literate.markdown("scripts/dining_philosophers_animation_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from ~/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation_iterate.jl
[ Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_animation_iterate.qmd
~/home/dgavdadev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_animation_iterate.qmd

julia> Literate.markdown("scripts/dining_philosophers_parametrized_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from ~/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_parametrized_iterate.jl
[ Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_parametrized_iterate.qmd
~/home/dgavdadev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_parametrized_iterate.qmd

julia> Literate.markdown("scripts/dining_philosophers_report_iterate.jl", "markdown"; flavor = Literate.QuartoFlavor())
[ Info: generating markdown page from ~/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report_iterate.jl
[ Info: writing result to ~/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_report_iterate.qmd
~/home/dgavdadev/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_report_iterate.qmd
```

Figure 9: Literate.notebook и Literate.markdown с QuartoFlavor

- 4 скрипта → 4 .ipynb + 4 .qmd/.html
- Все документы успешно сгенерированы и выполнены

Параметрическое исследование

Набор параметров

```
File Edit Selection View Go Run Terminal Help
dining_philosophers_report.jl dining_philosophers_report_iterate.jl dining_philosophers_parametrized_iterate.jl X
home > dgavdadev > 2026-1-study-simulation-modeling > labs > lab05 > project > scripts > dining_philosophers_parametrized_iterate.jl
1 # Параметрическое исследование сети Петри
2 #
3 # Сравнение поведения модели при разном количестве философов
4
5 using DrWatson
6 @quickactivate "project"
7 include(srcdir("DiningPhilosophers.jl"))
8 using .DiningPhilosophers
9 using DataFrames, CSV, Plots
10
11 ## Набор параметров
12
13 param_sets = [
14     (N = 3, tmax = 30.0),
15     (N = 5, tmax = 50.0),
16     (N = 7, tmax = 70.0),
17 ]
18
19 ## Проведение эксперимента
20
21 function run_experiment(N, tmax)
22     println("N = $N, tmax = $tmax")
23
24     net_classic, u0_classic, _ = build_classical_network(N)
25     df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
26     CSV.write(datadir("dining_classic_N$(N).csv"), df_classic)
27     dead_classic = detect_deadlock(df_classic, net_classic)
28
29     net_arb, u0_arb, _ = build_arbiter_network(N)
30     df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
31     CSV.write(datadir("dining_arbiter_N$(N).csv"), df_arb)
32     dead_arb = detect_deadlock(df_arb, net_arb)
33
34     return (N = N, tmax = tmax, deadlock_classic = dead_classic, deadlock_arbiter = dead_arb)
```

Figure 10: dining_philosophers_parametrized_iterate.jl

- param_sets: (N=3, tmax=30.0), (N=5, tmax=50.0), (N=7, tmax=70.0)
- Функция run_experiment(N, tmax) строит обе сети и проверяет deadlock


```

julia> include("scripts/dining_philosophers_parametrized_literate.jl")
N = 3, tmax = 30.0
N = 5, tmax = 50.0
N = 7, tmax = 70.0

=== Результаты экспериментов ===
3x4 DataFrame
  Row  N      tmax  deadlock_classic  deadlock_arbiter
   Int64 Float64      Bool              Bool
1     3   30.0      true              false
2     5   50.0      true              false
3     7   70.0      true              false
Warning: Assignment to 'N' in soft scope is ambiguous because a global variable by the same name exists: 'N' will be treated as a new local variable, overriding the existing global variable.
Warning: Assignment to 'eat_cols' in soft scope is ambiguous because a global variable by the same name exists: 'eat_cols' will be treated as a new local variable, overriding the existing global variable.
julia> @animate plot(parameter_composition)

```

Figure 11: Таблица deadlock_classic / deadlock_arbiter для N = 3, 5, 7

Результаты исследования

- Для всех N (3, 5, 7): deadlock_classic = true
- Для всех N (3, 5, 7): deadlock_arbiter = false

График: классическая сеть, N = 3, 5, 7

- N = 3 → тупик к $t \approx 0.7$ (1 импульс еды)
- N = 5 → тупик к $t \approx 17.5$ (несколько импульсов еды)
- N = 7 → тупик к $t \approx 6.5$ (2 импульса еды)

Заключение и выводы

Итоги работы

- Реализованы классическая сеть и сеть с арбитром для задачи обедающих философов
- Для N = 5 классическая сеть приходит к тупику ($t \approx 2.6$), сеть с арбитром работает без тупика на $t \in [0, 50]$
- Для N = 3, 5, 7 классическая сеть всегда приходит к тупику, арбитр — никогда
- Время до тупика в классической сети немонотонно: $t \approx 0.7$ (N=3), $t \approx 17.5$ (N=5), $t \approx 6.5$ (N=7)
- Построена анимация маркировки сети для N = 3

Освоенные навыки

- Реализация сетей Петри и проверка тупиковых ситуаций на Julia
- Стохастическое моделирование и сохранение результатов в CSV
- Построение анимаций (@animate, gif)

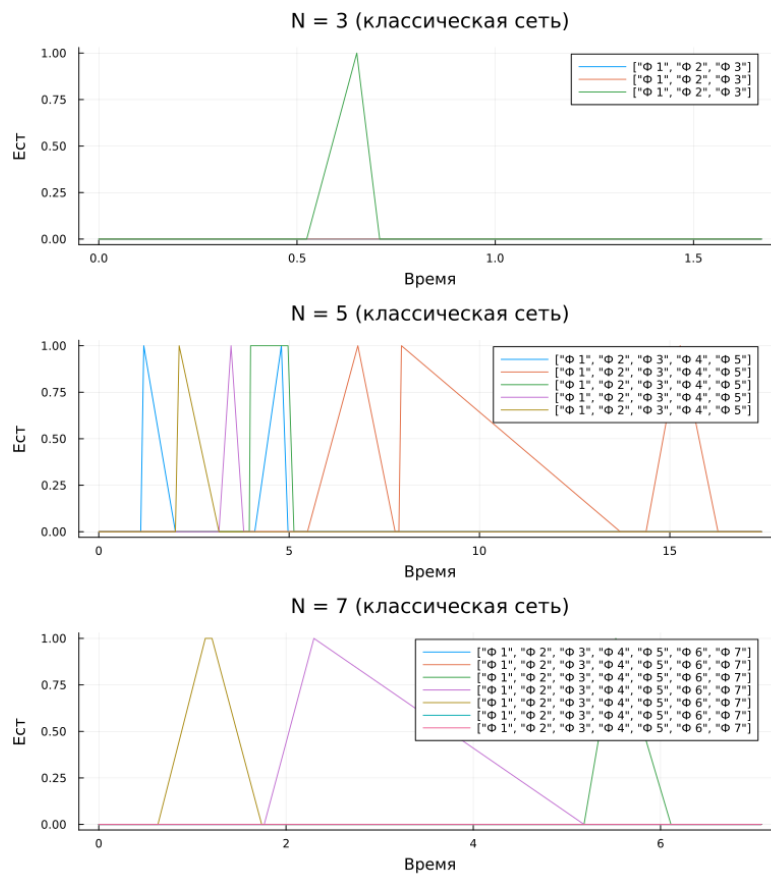


Figure 12: parameter_comparison.png

- Литературное программирование: `Literate.jl` → Jupyter Notebook и Quarto (QuartoFlavor)